# D

# Six MCPs I'd build next

> **Note**
>
> *Chapter 16 taught the build pattern. This appendix is the catalog of what to build next, given the converged stack the rest of the book installs (Notion as the canonical task/CRM/notes substrate; Gmail, Calendar, Slack, Buffer alongside it). Six worked sketches. No code. Each one is a prompt away from real.*

You won't build all six. You don't need to. What you need is to learn to recognize the *shape* of an MCP-shaped hole in your own week: the place where you keep manually pasting data from the same source, where Claude already does the thinking but is missing the wrapper. Once you see the shape, you can sketch your seventh, eighth, and ninth without me. The six below are the ones I would build next, in the order I'd build them.

A note on the canonical stack. By the time you finish the book, your installed MCPs include Gmail, Calendar, Notion (Iris's task router and Echo's CRM database both land here), Slack, Buffer, and the personal-data MCP from *Chapter 16*. The six sketches below are the ones that would *extend* that stack: close real gaps, unlock real jobs. If your day looks different and your stack does too (Linear, HubSpot, Pipedrive, Salesforce, Trello, ClickUp), the same shapes apply; the names are swap-ins, the rubric is the same.

## Spotting an MCP-shaped hole in your week

Three signals. Any one of them is enough to start sketching; two is enough to start building.

**Signal 1: "I keep manually pasting data from [X] into Claude."** Every Wednesday I exported a Stripe report and pasted the rows into Claude so Reuben could reconcile. The pasting was the symptom. The Stripe wrapper was the cure. If you're pasting the same shape of data from the same source more than once a week, you're paying a tax that an MCP can clear.

**Signal 2: "There's an API but no MCP."** The wrapped service publishes a real, documented API but no one has wrapped it for Claude Code yet. A weekend's worth of work, and the wrapper saves you forever.

**Signal 3: "Three different agents would all benefit from the same data shape."** This is the strongest signal. If Iris and Reuben and the morning brief all want the same query against the same source, that source earns an MCP. Single-use wrappers are skills (a 30-line file in `.claude/skills/`). Multi-use wrappers are MCPs. *Appendix C*'s rubric point 7, *could a 30-line skill replace this?*, is the test in the other direction.

The prompt to give Claude when you're not sure:

```
Here's what I do every week with [TOOL]: [DESCRIPTION].
Is this MCP-shaped? If yes, sketch the 3 tools the MCP would expose and who on my staff
would call each one. If no, tell me what would be a better wrapper (skill, hook, slash
command) and why.
```

The "no" answer is as useful as the "yes" answer. An MCP earns its existence; not everything does.

# MCP 1: Bank CSV importer

**What it wraps:** A folder of monthly bank and credit-card CSVs that the user drops in by hand. The MCP doesn't talk to a bank API: it just reads the files in `~/work/finance/statements/` and exposes them as structured tools. Easiest of the six because there's no live auth.

**Three tools it would expose:**

- `import_statement(path)`, read a CSV, normalize the columns, return the rows
- `categorize_transactions(month)`: apply the user's category rules to that month's rows
- `get_balance_at_date(date)`: return the running balance across all statements as of a given date.

**Who would benefit:** Reuben, the bookkeeper from *Chapter 13*. Today Reuben asks the user to drop a CSV and then parses it inline. With this MCP, Reuben just calls `import_statement` and gets typed rows. The Friday reconciliation job (*Appendix A* job 3) becomes one tool call instead of three.

**The unlock:** You can ask, *"what did I spend on contractors last quarter?"* and the answer arrives in seconds, with the line items, without re-pasting anything. Reuben stops being a CSV parser and starts being an analyst.

**Gotchas:** Bank CSV formats drift. Add a column-detection step in `import_statement` that errors loudly if the header doesn't match what it saw last month. Don't silently re-map: surface the change.

**Build prompt:**

```
Build an MCP server at ~/work/mcp-servers/bank-csv/. Three tools:
import_statement(path), categorize_transactions(month), get_balance_at_date(date).
Reads CSVs from ~/work/finance/statements/. Pattern-match Chapter 16's MCP for shape. Add
a smoke test that exits non-zero if any column header changed since last run.
```

# MCP 2: Notion power tools

**What it wraps:** The Notion database the rest of the book leans on (Iris's Tasks DB, Echo's CRM database, the Project Notes pages). The official Notion MCP covers the basics; this one extends it for the queries the staff actually keeps writing.

**Three tools it would expose:**

- `bulk_archive(database_id, filter)`, archive rows matching a filter without one-by-one calls

- `cross_database_lookup(source, target, key)`: join two Notion databases by a shared key (a contact in the CRM linked to a task in the Tasks DB)
- `find_overdue_followups(threshold_days)`: return rows where last-interaction-date exceeds the threshold.

**Who would benefit:** Echo, the CRM hygienist from *Chapter 14*. Today Echo can read and write rows but can't efficiently join Tasks against CRM contacts; the join lives in the agent prompt. With this MCP, Echo asks *"who haven't I emailed in 30 days who has a Task assigned?"* and gets the answer in one call.

**The unlock:** Notion stops being a place where each agent independently re-derives the same query. The MCP becomes the canonical join layer; agents call it instead of re-implementing it.

**Gotchas:** Notion's API rate-limits per integration. Multiple agents hitting cross-database lookups can collide. Build in a brief retry with backoff. Schema changes in Notion (renaming a column) break the wrapper silently: add a schema check.

**Build prompt:**

```
Build an MCP server at ~/work/mcp-servers/notion-power/. It wraps the Notion databases
listed in ~/work/.claude/notion-databases.json. Three tools: bulk_archive,
cross_database_lookup, find_overdue_followups. Use the Notion API token from the keychain
entry "notion-power". Add a schema check that runs on startup and errors if any expected
column is missing.
```

# MCP 3: Stripe revenue summarizer

**What it wraps:** The Stripe API for a one-person business, payments, customers, subscriptions, refunds. Read-only. Real money is on the other side, so the auth scope is tighter.

**Three tools it would expose:**

- `monthly_revenue(month)`, gross, net, refunds, by product line
- `failed_payments_last_week()`: list of failed charges with customer and reason
- `top_customers_by_ltv(n)`: ranked list, scoped to active customers only.

**Who would benefit:** Reuben. Today the user pastes a Stripe report into Claude every Friday so the P&L shows revenue. With this MCP, Reuben produces a P&L that includes revenue without manual entry. A `SessionEnd` hook on Friday afternoons can post the week's revenue note to the user's journal.

**The unlock:** Reconciliation that includes the income side, not just the expense side. Reuben stops needing the user as a courier between Stripe and itself.

**Gotchas:** Real API keys, real money. Use a *restricted* Stripe key with read-only scopes: the API supports it. Never read-write. Note the auth complexity in your `CLAUDE.md` so you remember the scope when an agent tries to do something the key can't.

**Build prompt:**

```
Build an MCP server at ~/work/mcp-servers/stripe/. Read-only — restricted key only. Three
tools: monthly_revenue, failed_payments_last_week, top_customers_by_ltv.
Pull the key from the keychain entry "stripe-readonly". Reject any tool call that
attempts a write. Add a smoke test that confirms the key cannot create or delete.
```

# MCP 4: Calendar conflict resolver

**What it wraps:** Google Calendar, but with intent-aware scheduling. The Calendar MCP from *Chapter 7* reads events; this one resolves them.

**Three tools it would expose:**

- `find_open_slot(duration, preferences)`, find a slot matching length and preferences (mornings only, no lunch, prefer Tuesdays)
- `propose_reschedule(event_id, reason)`: given an event, propose three new times that fit the attendees' preferences
- `detect_double_books(date_range)`: surface overlaps the user might not have noticed

**Who would benefit:** Iris. Today Iris drafts a reschedule reply by guessing at slots; with this MCP she checks the calendar first and proposes real ones. Cuts the *"let me get back to you on times"* loop in half.

**The unlock:** Iris stops being a polite vague-time-suggester and becomes someone who actually proposes calendar slots. The friction of every reschedule drops. Different from the read-only Calendar MCP because it resolves intent, not just queries data.

**Gotchas:** Time zones. Always store and return UTC; convert at the edge. Daylight saving transitions break naive duration math; use a real datetime library (Claude will pick one).

**Build prompt:**

```
Build an MCP server at ~/work/mcp-servers/calendar-resolver/. Three tools:
find_open_slot, propose_reschedule, detect_double_books. Wraps Google Calendar via the
existing OAuth credentials. Use UTC internally; convert at tool boundaries. Add a test
that confirms a DST transition doesn't shift a 60-minute slot.
```

# MCP 5: Slack thread summarizer

**What it wraps:** The Slack API for cross-channel synthesis. Goes beyond the read-thread primitive: it summarizes, ranks, and digests.

**Three tools it would expose:**

- `summarize_thread(thread_id)`, one-paragraph synthesis of a long thread

- `find_unanswered_pings(user, hours)`: surface DMs and mentions the user hasn't replied to
- `digest_channel(channel_id, hours)`: produce a Tuesday-morning digest of a channel the user can't keep up with.

**Who would benefit:** The morning brief (*Chapter 9*). Today the brief reads Gmail, Calendar, Notion. Adding Slack closes the obvious gap. The weekend-digest pattern works without you opening Slack at all.

**The unlock:** Slack becomes an input to your staff instead of an interruption to your day. The morning brief tells you what crossed Slack overnight; the digest tells you what crossed channels you don't actively follow. The "I haven't been on Slack in three days" anxiety goes away because something else has been.

**Gotchas:** Slack rate-limits per channel and per workspace. Stagger calls. The digest tool can blow through tokens fast if it pulls a hundred messages: cap the input window and summarize incrementally. Cross-reference Appendix B.

**Note:** Anthropic's marketplace may ship an official Slack summarizer before this book reaches second print. Check before building. If it exists, install per Appendix C; otherwise, build.

**Build prompt:**

```
Build an MCP server at ~/work/mcp-servers/slack-summarizer/. Three tools:
summarize_thread, find_unanswered_pings, digest_channel. Use the Slack token from
keychain entry "slack-readonly". Cap input window at the most recent 200 messages per
call to control token cost.
```

# MCP 6: Screenshot-to-spec converter

**What it wraps:** A vision model plus a structured-output schema. The wrapped service is itself an LLM, which makes the failure modes different from the other five.

**Three tools it would expose:**

- `extract_ui_text(image_path)`, pull every visible string from a screenshot
- `describe_layout(image_path)`: produce a structured description of the layout (header, sidebar, three cards, footer)
- `generate_clone_prompt(image_path)`: produce a prompt-to-Claude that, fed back, would generate HTML matching the screenshot

**Who would benefit:** Elena's design-review use case, point at a competitor's screenshot and get a structured spec. Also useful for the writer who screenshots a page they like and wants the bones of it for a quick prototype. Plays into *Chapter 22* (the website) and *Chapter 23* (browser + computer use) directly.

**The unlock:** Design references stop being images you paste into a conversation and become structured specs other agents can act on. The newsroom can lay out a piece against a clone-prompt; Iris can describe a confusing UI screenshot a colleague sent.

**Gotchas:** The hardest of the six. Vision models hallucinate confidently: the layout description that looks structured may have invented a sidebar that isn't there. Pair every output with the original image when handing it to a downstream agent. Treat the structured output as a *draft* spec, not ground truth.

**Build prompt:**

```
Build an MCP server at ~/work/mcp-servers/screenshot-to-spec/. Three tools:
extract_ui_text, describe_layout, generate_clone_prompt. Wraps a vision model via the API
key in keychain entry "vision-api". Every tool returns the original image path alongside
its structured output, so downstream agents can verify. Add a test that runs against
three known screenshots and asserts the layout count matches.
```

# A closing wedge

Build the one that would change your week the most. My order, given lift vs. cost for a single operator: **bank CSV importer first** (one weekend, immediate Reuben unlock), **Slack summarizer second** (biggest morning-brief upgrade), **Notion power tools third** (biggest cross-agent win, since Iris and Echo both benefit). Stripe, calendar conflicts, and screenshot-to-spec are real wins but real builds.

If your stack swaps Notion for Linear/HubSpot/Salesforce or Slack for Teams/Discord, the shapes carry: replace the names, the tools stay. The catalog is six sketches; the discipline is the signal-spotting at the top of this appendix. Once you see one MCP-shaped hole, you'll see them everywhere. Chapter 16 is the pattern; this appendix is the catalog. By your third build you'll be sketching without a template.